Automated Web Application Builder using Large Language Models

A project report submitted in partial fulfilment of the requirements for the award of

Degree of Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

By

P. SAI MANOHAR (22FE1A4347)

A. SUJAN (22FE1A4302)

P. SAI MUNEESH (22FE1A4343)

K. GOPI (22FE1A4329)

Under the guidance of

Dr. U. Usha Rani, M. Tech, PhD

Professor

Department of Allied Computer Science and Engineering

VIGNAN'S LARA INSTITUTE OF TECHNOLOGY & SCIENCE

(AUTONOMOUS)

Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada

Accredited by NAAC 'A+' and NBA | ISO 9001 : 2015

Vadlamudi - 522 213, Guntur District

VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)
Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by NAAC 'A+' and NBA | ISO 9001 : 2015
Vadlamudi - 522 213, Guntur District

CERTIFICATE

This is to certify that the project report entitled **"AUTOMATED WEB APPLICATION BUILDER USING LARGE LANGUAGE MODELS"** is a bonafide work done by **P. SAI MANOHAR (22FE1A4347), A. SUJAN (22FE1A4302), P. SAI MUNEESH (22FE1A4343), K. GOPI (22FE1A4329),** under my guidance and submitted in fulfilment of the requirements for the award of the degree of Bachelor of Technology in **COMPUTER SCIENCE AND ENGINEERING** from **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA**. The work embodied in this project report is not submitted to any University or Institution for the award of any Degree or diploma.

Project Guide

Dr. U. Usha Rani, M.Tech, PhD

Professor

Head of the Department

Dr. G. Rajesh Chandra, M.Tech, PhD

Professor

External Examiner

DECLARATION

We here by declare that the project report entitled **"AUTOMATED WEB APPLICATION BUILDER USING LARGE LANGUAGE MODELS"** is a record of an original work done by us under the guidance of **Dr. U. USHA RANI,** Professor of Computer Science and Engineering and this project report is submitted in the fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering. The results embodied in this project report are not submitted to any other University or Institute for the award of any Degree or Diploma.

PROJECT MEMBERS	**SIGNATURE**
P. SAI MANOHAR (22FE1A4347)	_____
A. SUJAN (22FE1A4302)	_____
P. SAI MUNEESH (22FE1A4343)	_____
K. GOPI (22FE1A4329)	_____

Place: Vadlamudi.

Date:

ACKNOWLEDGEMENT

The satisfaction that accompanies with the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success.

We are grateful to **Dr. U. USHA RANI,** Professor, Department of Computer Science and Engineering for guiding through this project and for encouraging right from the beginning of the project till successful completion of the project. Every interaction with him was an inspiration.

We thank **Dr. G. RAJESH CHANDRA**, Professor & HOD, Department of Computer Science and Engineering for support and valuable suggestions.

We also express our thanks to **Dr. K. PHANEENDRA KUMAR**, Principal, Vignan's Lara Institute of Technology & Science for providing the resources to carry out the project.

We also express our sincere thanks to our beloved **Chairman Dr. LAVU RATHAIAH** for providing support and stimulating environment for developing the project.

We also place our floral gratitude to all other teaching and lab technicians for their constant support and advice throughout the project.

Project Members

P. SAI MANOHAR	(22FE1A4347)
A. SUJAN	(22FE1A4302)
P. SAI MUNEESH	(22FE1A4343)
K. GOPI	(22FE1A4329)

Table of Contents

ABSTRACT

The development of modern web applications has become increasingly complex due to the need to coordinate user interfaces, backend logic, data persistence, and deployment infrastructure. Existing AI-assisted and low-code development frameworks largely emphasize frontend generation or template driven design, often producing loosely connected code fragments with limited backend functionality, minimal database integration, and insufficient support for validation and real-world deployment, which reduces reliability and increases manual intervention. This paper presents an automated web application builder that leverages Large Language Models, specifically Google Gemini, to transform natural language requirements into complete, integrated, and deployable full-stack web applications. The proposed system adopts a structured generation pipeline that enforces architectural consistency across frontend, backend, and database layers by integrating a Python–Flask backend, RESTful APIs, persistent data storage, live preview–based validation, and iterative refinement. Additionally, automated deployment configuration enables direct cloud deployment without manual setup. Experimental evaluation demonstrates an approximate functional accuracy of 86%, measured by the successful execution of applications with integrated frontend, backend, database connectivity, and cloud deployment readiness. These results indicate that addressing structural and deployment limitations through validation-driven and deployment-aware workflows significantly improves reliability, making the proposed system suitable for academic projects, rapid prototyping, and practical full-stack web application development.

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing
API	Application Program Interface
UI	User Interface
REST	Representational State Transfer
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DB	Database
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
GPU	Graphics Processing Unit
URL	Uniform Resource Locator
CRUD	Create, Read, Update, Delete

CHAPTER 1

INTRODUCTION

The rapid growth of digital technologies has significantly increased the demand for web applications across various sectors including education, business, healthcare, and government services. Web applications play an important role in enabling communication, data management, and service delivery through online platforms. As organizations continue to digitize their operations, the need for efficient and scalable web application development has become increasingly important.

Modern web applications are built using a combination of multiple technologies that operate across different layers of the software architecture. These layers typically include the frontend interface, backend processing logic, database management systems, and deployment infrastructure. The frontend layer is responsible for presenting the user interface and handling user interactions, while the backend manages application logic, data processing, and communication with databases. Additionally, deployment platforms and cloud services are required to host applications and ensure accessibility through the internet.

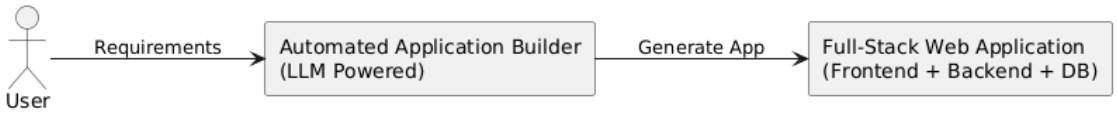


Fig. 1.1 Automated Application Builder Overview

Developing a complete web application therefore requires expertise in several programming languages, frameworks, and development tools. For many developers, particularly beginners and students, coordinating these different components can be complex and time-consuming. Even relatively simple applications require writing large amounts of boilerplate code, configuring development environments, integrating databases, and managing deployment processes.

In recent years, advances in artificial intelligence have introduced new possibilities for simplifying software development processes. Large Language Models (LLMs) have emerged as powerful tools capable of understanding natural language instructions and generating structured programming code. These models are trained on large datasets containing natural language and programming examples, enabling them to assist developers with coding tasks.

By integrating LLMs with automated development frameworks, it becomes possible to generate full-stack web applications directly from natural language descriptions provided by users. This approach significantly reduces manual coding effort and enables rapid prototyping of applications. The Automated Web Application Builder proposed in this project aims to leverage

these capabilities to simplify the process of full-stack web development.

1.1 Full-Stack Web Application Development Challenges

Full-stack web development involves the design, implementation, and integration of multiple components required to create a fully functional web application. These components include the frontend user interface, backend application logic, database management systems, and deployment infrastructure. Each component requires specialized tools and technologies, making the development process complex and resource intensive.

The frontend layer is responsible for presenting information to users through web browsers. Technologies such as HTML, CSS, and JavaScript are commonly used to create interactive user interfaces. Developers must design layouts, manage user interactions, and ensure compatibility across different devices and browsers. Achieving responsive and visually appealing designs requires both programming skills and design knowledge.

The backend layer handles server-side operations such as data processing, authentication, and communication with databases. Backend development typically involves frameworks such as Flask, Node.js, or Django. Developers must design application logic, implement APIs, and manage data flow between the frontend and the database. This requires understanding server architecture, security practices, and software design principles.

Database management is another critical component of web applications. Databases are used to store and retrieve data efficiently. Developers must design data models, manage database queries, and ensure data integrity. Modern applications often rely on cloud-based databases that require additional configuration and integration with backend services.

Deployment and hosting represent another layer of complexity. Once an application is developed, it must be deployed to a server or cloud platform to make it accessible to users. This process may involve configuring servers, managing dependencies, and ensuring application scalability.

Because of these multiple layers, full-stack development can be challenging for individuals with limited experience. Beginners often struggle to integrate frontend, backend, and database components into a cohesive system. As a result, there is a growing need for tools that simplify and automate the web application development process.

1.2 Large Language Models in Software Development

Large Language Models (LLMs) are advanced artificial intelligence systems designed to process and generate human-like language. These models are typically built using deep learning techniques and transformer-based neural network architectures. LLMs are trained on massive datasets containing text from books, websites, research papers, and programming code.

The primary capability of LLMs is their ability to understand context and generate meaningful responses based on input prompts. This makes them useful for a wide range of natural language processing tasks such as text generation, summarization, translation, and question answering. In recent years, LLMs have also demonstrated strong capabilities in generating programming code.

LLMs are now widely used in AI-powered coding assistants that help developers write code more efficiently. They can generate functions, explain code segments, suggest improvements, and even detect potential errors. By automating repetitive programming tasks, these tools improve developer productivity and reduce development time.

However, generating code snippets alone is not sufficient for building complete applications. Effective application development requires coordination across multiple components such as frontend interfaces, backend services, and databases. Therefore, LLMs must be integrated within structured frameworks that guide them to generate complete application architectures.

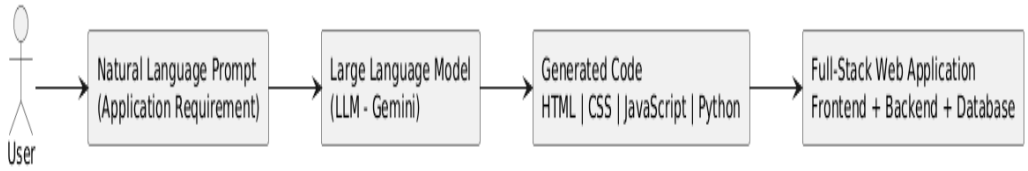


Fig 1.2 Large Language Model for Code Generation

1.3 AI-Based Code Generation for Web Applications

AI-based code generation refers to the process of automatically producing programming code using artificial intelligence models. Instead of manually writing code, developers can provide descriptions of the desired functionality, and AI systems generate the corresponding program code.

Recent advancements in machine learning and natural language processing have enabled the development of powerful AI models capable of generating high-quality code. These models analyze the structure of programming languages and learn patterns from large datasets of source code.

AI code generation tools have become increasingly popular in modern software development environments. Many integrated development environments now incorporate AI-based assistants that provide code suggestions and automate routine tasks. These systems help developers generate boilerplate code, create user interfaces, and implement standard programming patterns.

In the context of web application development, AI models can generate components such as HTML structures, CSS styling rules, JavaScript logic, and backend server code. By automating these tasks, developers can significantly reduce the time required to build applications.

Despite these advantages, many existing AI coding tools focus only on generating individual code fragments rather than complete applications. Developers often need to manually integrate the generated components, configure databases, and manage deployment settings.

The proposed Automated Application Builder addresses this limitation by generating complete application structures that include frontend interfaces, backend services, and database integration. This ensures that the generated code is organized, consistent, and ready for execution.

1.4 Automated Application Development using LLMs

Automated application development refers to the use of intelligent systems to generate software applications with minimal manual programming effort. The goal of automated development tools is to simplify the software development process and make application creation accessible to a wider range of users.

Large Language Models have significantly improved the capabilities of automated development platforms. By understanding natural language descriptions of application requirements, these models can generate structured programming code that implements the desired functionality.

Automated development systems typically follow a pipeline that includes requirement interpretation, code generation, validation, and deployment. The system first analyzes the user's description of the application and identifies key components such as user interface elements, backend services, and database requirements.

Automated application development systems are particularly useful for rapid prototyping, educational projects, and early-stage software development. They allow users to quickly transform conceptual ideas into working applications without requiring extensive programming expertise.

The integration of LLMs into automated development frameworks has opened new possibilities for building intelligent systems capable of generating complex applications from simple descriptions.

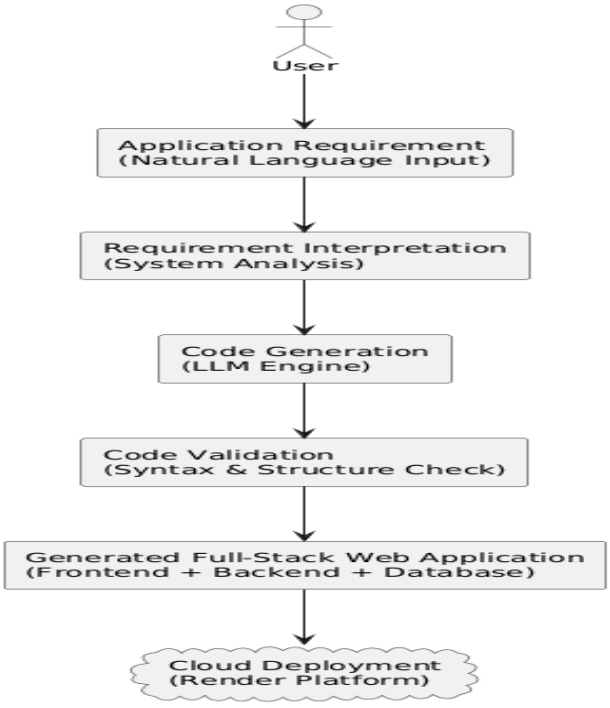


Fig 1.3 Automated Application Development using LLMs

1.5 Working of the Automated Application Builder

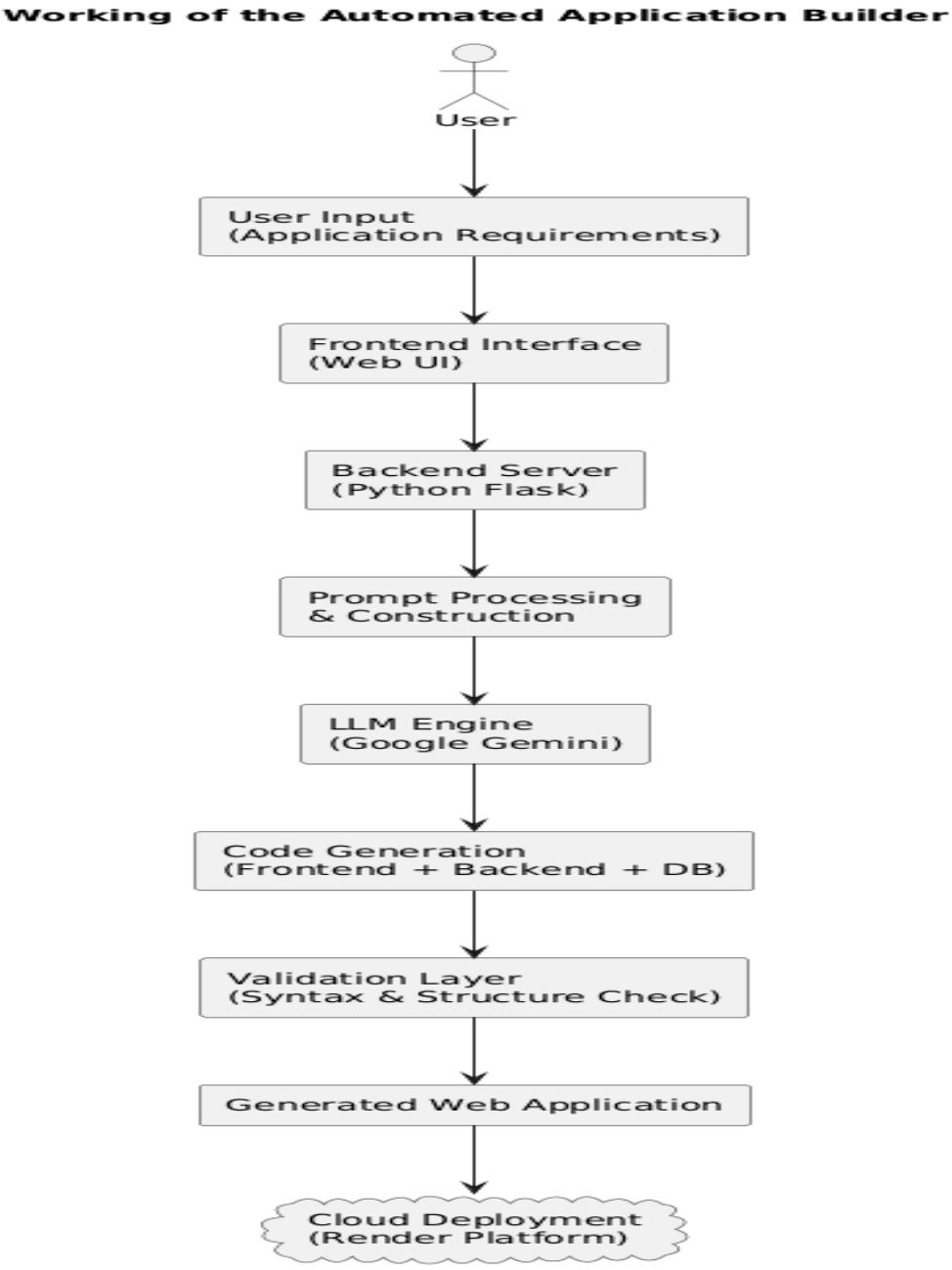


Fig. 1.4 Working of the Automated Application Builder

CHAPTER 2
LITERATURE SURVEY

2.1 JOURNALS

2.1.1 Sketch-Based UI Design

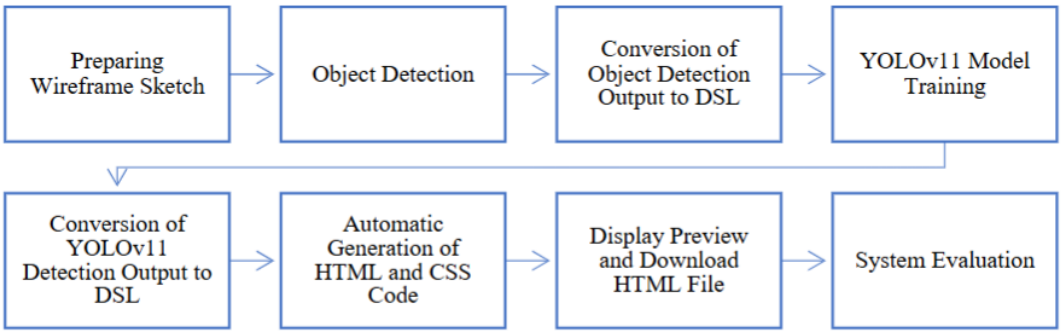


Fig. 2.1.1 Architecture of Sketch based HTML Code Generation

AI-based code generation has become an important research area in software engineering. Several studies have explored how artificial intelligence can generate programming code automatically based on user input or design specifications. These systems aim to reduce the amount of manual coding required to build software applications.

One approach involves generating user interface code from graphical designs or sketches. Techniques such as deep learning and computer vision have been used to convert graphical user interface images into HTML and CSS code. For example, the pix2code system uses convolutional neural networks to translate GUI screenshots into frontend code. Similarly, other research studies have proposed methods that convert hand-drawn sketches into working web page layouts.

These approaches significantly reduce the time required to develop frontend interfaces. However, most AI-based code generation systems focus mainly on frontend components and do not address backend logic, database management, or deployment configuration. As a result, developers must manually integrate the generated components to build a complete application

2.1.2 Template-Based UI Design

Template-based UI design systems focus on generating user interfaces using predefined layouts and reusable design components. These systems reduce design effort by providing ready-made

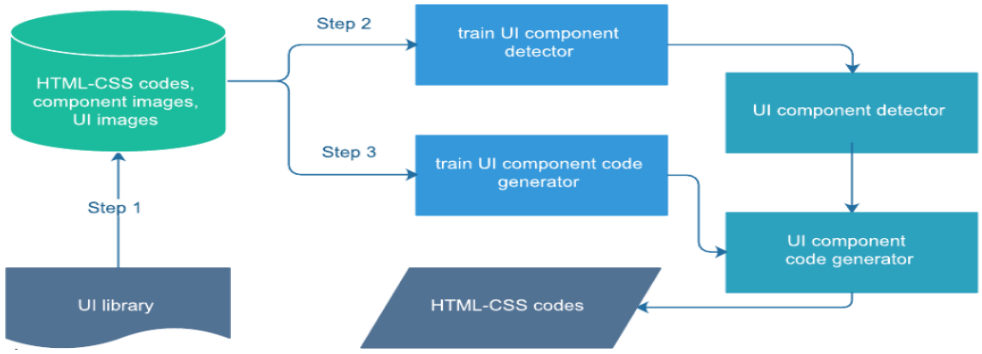


Fig. 2.1.2 Architecture of the Template-Based UI Design System

-templates for common application screens such as dashboards, login pages, forms, and navigation panels.

Such systems typically allow users or developers to select a suitable template and customize it based on requirements. They can dynamically adjust elements like colors, typography, layout structure, and content placement while maintaining consistency in design standards. Many frameworks also support responsive design, ensuring compatibility across different devices.

Although template-based UI design simplifies the interface creation process, it may still require manual adjustments to meet specific user needs. Developers often need to refine layouts, integrate backend functionality, and ensure a seamless user experience. Therefore, while template-based approaches significantly speed up UI development, complete automation of design customization remains a challenge.

2.1.3 Low-Code and No-Code Platforms

Low-code and no-code platforms have emerged as popular tools for simplifying software development. These platforms allow users to build applications using graphical interfaces, drag-and-drop components, and predefined templates rather than writing large amounts of code.

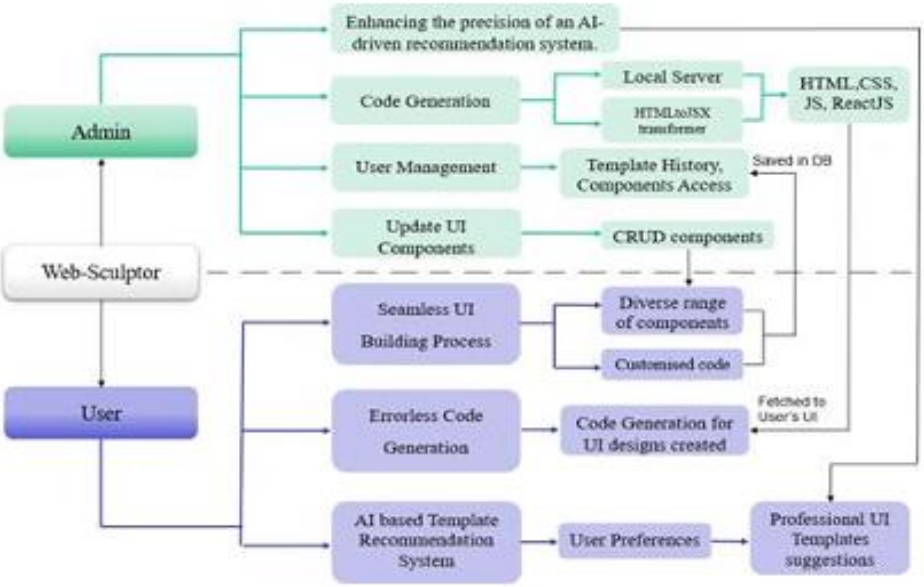


Fig. 2.1.3 Architecture of Low Code and No Code Platforms

Examples of such platforms include systems that provide visual editors for designing application interfaces and workflows. Users can connect components and configure logic using visual tools, enabling rapid development of simple applications.

Although low-code platforms are useful for quick prototyping, they often impose limitations on customization and scalability. Developers may find it difficult to implement complex backend logic, database operations, or deployment workflows using these platforms. Consequently, low-code solutions are often unsuitable for research projects or applications requiring flexible system architectures.

2.1.4 LLM-Based Programming Assistants

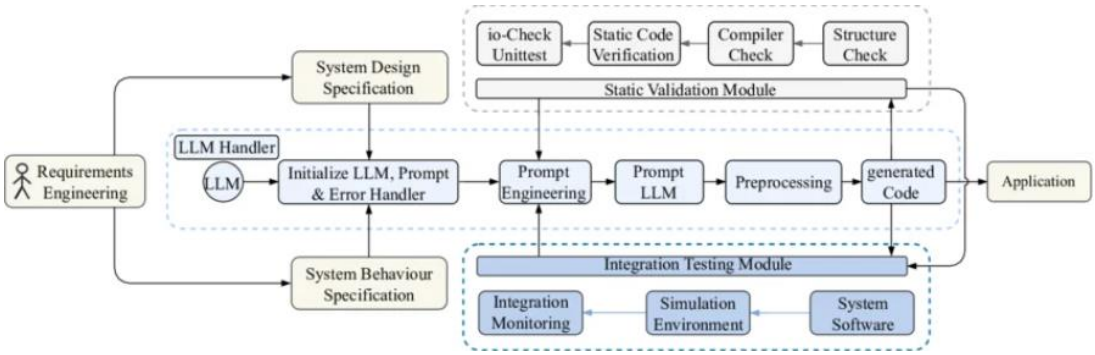


Fig. 2.1.4 Architecture of LLM-Based Programming Assistants

Large Language Models have introduced new possibilities for intelligent programming assistance. LLM-based coding tools can analyze natural language instructions and generate programming code in multiple languages. These systems are widely used in development environments to assist programmers with writing code more efficiently.

LLM-based programming assistants can generate code snippets, suggest improvements, explain programming concepts, and help identify potential errors in source code. By learning patterns from large datasets of programming examples, these models can reproduce complex programming structures.

However, many LLM-based tools operate primarily as chat-based assistants and generate code fragments rather than complete applications. Developers must still manually integrate the generated code into their projects and configure system components such as databases and deployment environments.

This limitation highlights the need for systems that integrate LLM capabilities within a structured development pipeline capable of generating complete full-stack applications.

2.1.5 Prompt Engineering for Code Generation

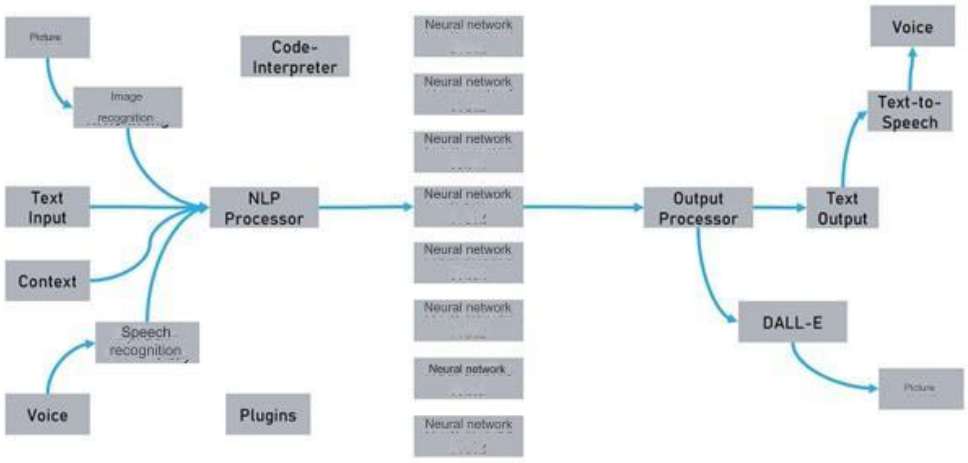


Fig. 2.1.5 Architecture of Prompt Engineering for Code Generation

Prompt engineering is an important technique used to guide the behavior of Large Language Models. A prompt refers to the input instruction provided to the model that specifies the task to be performed. Designing effective prompts can significantly improve the accuracy and relevance of the generated output.

In the context of AI-based code generation, prompt engineering is used to instruct the model to generate specific programming structures, frameworks, and system architectures. Structured prompts may include information about required technologies, coding standards, and deployment

constraints.

Researchers have demonstrated that carefully designed prompts can improve the consistency of generated code and reduce errors. Some systems also incorporate iterative prompt refinement mechanisms, where the model output is evaluated and prompts are adjusted to improve results.

Prompt engineering therefore plays a critical role in ensuring that AI models generate reliable and functional application code.

2.2 Existing System

Existing systems for automated web development primarily focus on individual stages of the software development lifecycle. Some systems specialize in frontend generation, while others generate code snippets based on natural language descriptions.

Frontend automation frameworks can convert graphical designs or sketches into HTML code, but they usually do not generate backend services or database integration. Similarly, AI coding assistants can produce program code from textual descriptions, but they often generate isolated code fragments rather than complete applications.

Low-code platforms attempt to simplify development using visual tools and templates. However, these platforms often restrict customization of backend logic and database structures, making them unsuitable for complex applications.

Because of these limitations, developers frequently need to manually combine generated components and configure deployment environments. This process reduces the effectiveness of existing automation tools.

2.3 Limitations of Existing Methods

Although many systems aim to simplify web application development, several limitations remain in current approaches.

First, many AI-based frameworks focus primarily on frontend generation and do not provide full-stack integration. As a result, developers must manually implement backend logic and database connections.

Second, many natural language code generation systems generate code fragments without ensuring that the generated components work together as a complete application. This often leads to inconsistencies and runtime errors.

Third, existing tools typically lack integrated validation mechanisms and deployment support. Developers must manually test generated code and configure deployment environments before the application can be executed.

These limitations highlight the need for a comprehensive framework that integrates requirement interpretation, AI-based code generation, validation, database management, and deployment within a single automated pipeline. The proposed Automated Web Application Builder addresses these challenges by providing a structured workflow that generates complete and deployable web applications.

CHAPTER 3
METHODOLOGY

3.1 Proposed System

The proposed system is designed to automate the process of generating full-stack web applications using natural language input. Instead of manually writing frontend and backend code, users can describe the application they want to build, and the system automatically generates the required project structure.

The core idea of the system is to combine the capabilities of **Large Language Models with a structured development pipeline**. The system interprets user requirements, generates application code using the Gemini model, validates the generated output, and organizes the code into a deployable project structure.

The backend server is implemented using the **Flask web framework**, which provides API endpoints for processing user requests. The backend communicates with the Gemini API to generate application code and performs additional processing to ensure that the generated code follows predefined architectural constraints.

The generated application includes the following components:

- Frontend user interface
- Client-side JavaScript functionality
- Backend API services
- Project configuration files
- Deployment configuration

By automating these steps, the system reduces development complexity and enables rapid generation of working web applications.

3.1.1 Advantages of the Proposed System

The proposed system provides several advantages compared to traditional web development methods.

First, the system significantly reduces manual coding effort by automatically generating

application code using a Large Language Model. Developers do not need to write repetitive boilerplate code for basic application components. Instead, the AI model generates the required code structure based on user input.

Second, the system enables rapid prototyping of web applications. Users can quickly generate working applications by describing their requirements in natural language. This is particularly useful in educational environments where students need to develop prototypes within limited time.

Another important advantage is the system's ability to enforce architectural consistency. The prompt used for code generation contains strict guidelines that ensure the generated application follows a consistent design pattern. This improves code readability and maintainability.

The system also includes built-in validation mechanisms that analyze the generated code and detect structural errors before deployment. This helps prevent runtime issues and improves the reliability of generated applications.

Furthermore, the automated project structure generation ensures that generated applications follow a standardized folder organization. This makes it easier for developers to understand and modify the generated code if necessary.

Finally, the system supports automated deployment preparation, allowing generated applications to be deployed to cloud hosting platforms such as Render without manual configuration.

3.2 System Workflow

The Automated Application Builder follows a structured workflow that transforms natural language input into a deployable web application. The workflow consists of several interconnected stages that ensure the generated code is valid, organized, and ready for deployment.

The workflow begins when the user submits application requirements through the system interface. These requirements include information about the application type, functionality, and desired features.

The backend server processes the request and constructs a detailed prompt that is sent to the Gemini Large Language Model. The AI model analyzes the prompt and generates complete application code including HTML, CSS, and JavaScript.

After the code is generated, the system performs several validation checks to ensure that the

output follows predefined architectural rules. If the code passes validation, it is organized into a structured project directory containing frontend files, backend logic, and configuration files.

Finally, the system prepares the generated project for deployment by copying the project files into a deployment directory and generating necessary configuration files required for hosting the application.

This automated workflow ensures that users can generate and deploy web applications with minimal manual intervention.

3.2.1 Requirement Acquisition

The process begins when the user provides application requirements through the frontend interface. The user specifies information such as:

- Project name
- Application type
- Description of the application
- Preferred technology stack
- Backend framework

This information is sent to the Flask backend using a REST API request. The backend receives the request in the /api/generate endpoint and extracts the relevant data fields.

The backend then prepares the input data for prompt generation by organizing the user requirements into structured variables.

3.2.2 Prompt Construction

After receiving user input, the backend constructs a detailed prompt that is sent to the Gemini Large Language Model. Prompt construction is a critical component of the system because it determines the quality and structure of the generated code.

The prompt includes detailed instructions that guide the AI model in generating a complete web application. These instructions specify the following requirements:

- Generate a single-page HTML application
- Include embedded CSS and JavaScript
- Implement responsive design
- Use semantic HTML elements
- Avoid external libraries

- Implement smooth scrolling navigation
- Include fallback logic for API failures

The prompt also enforces strict rules to ensure that the generated application does not include unsafe navigation links or unnecessary external resources.

By defining clear constraints and requirements in the prompt, the system ensures that the generated code follows a consistent architecture.

3.2.3 AI-Based Code Generation

Once the prompt is constructed, it is sent to the Gemini API using the Google Generative AI Python library. The Gemini model processes the prompt and generates the requested application code.

The generated output typically contains a complete HTML document that includes embedded CSS and JavaScript. The HTML document defines the structure of the web page, while the CSS section defines styling rules and the JavaScript section implements interactive functionality.

The AI model also generates user interface components such as navigation menus, content sections, interactive buttons, and responsive layouts.

The generated code is returned to the Flask backend as a text response. The backend then processes the response to remove unnecessary formatting such as markdown code fences.

After cleaning the generated output, the backend extracts the HTML, CSS, and JavaScript components from the generated code.

3.2.4 Code Validation

After the code generation stage, the system performs several validation checks to ensure that the generated application is structurally correct and ready for execution. The purpose of the validation module is to analyze the generated code and verify that it follows predefined structural and functional requirements.

The validation process includes several checks:

• Verification of required HTML tags

• Detection of empty section elements

• Verification of style and script blocks

• Validation of API fetch fallback logic

To evaluate the correctness of the generated code, a validation score can be defined based on the

number of successfully passed validation checks.

$$Validation\ Score = N\_passed/N\_total \times 100$$

where:Type equation here.

- N_{passed}= number of validation checks successfully passed
- N_{total}= total number of validation checks performed

This score represents the structural quality of the generated application code.

Another important metric used in the system is **HTML structure completeness**, which verifies whether all required HTML elements are present in the generated document.

$$HTML\ Completeness = T\_present/T\_required \times 100$$

where:

- $T_{present}$= number of required HTML tags present in the generated code
- $T_{required}$= total number of mandatory HTML tags

The system also evaluates **section integrity** to ensure that generated sections are not empty. This can be expressed as:

$$Section\ Integrity = S\_nonempty/S\_total \times 100$$

where:

- $S_{nonempty}$= number of sections containing valid content
- S_{total}= total number of sections generated by the model

If any section fails this validation, the system flags the output for correction or regeneration.

The system also enforces several security and structural rules. For example, all <img> tags are replaced with placeholder elements to avoid external dependencies. Similarly, root navigation links and unsafe navigation behaviors are removed to prevent unwanted page reloads or navigation loops.

These validation mechanisms improve the reliability of generated applications and ensure that the output follows predefined development guidelines. By integrating automated validation within the generation pipeline, the system reduces runtime errors and ensures that the generated web application maintains structural consistency and usability.

3.2.5 Project File Generation

Once the generated code passes validation, the backend organizes the code into a structured

project directory. This step ensures that the generated application follows a standard file organization pattern.

The project directory typically contains the following files:

```
generated_projects/
    project_name/
        index.html
        style.css
        app.js
        server.py
        requirements.txt
        render.yaml
```

The HTML file contains the main application interface. The CSS file contains styling rules, and the JavaScript file contains client-side logic.

The backend code provides API endpoints that allow the application to handle data requests. Additional configuration files are generated to support deployment on cloud hosting platforms.

3.2.6 Deployment Preparation

The final stage of the system workflow is deployment preparation. In this stage, the system copies the generated project files into a deployment directory that will be used for hosting the application.

The deployment module prepares configuration files required by the hosting platform. For example, the system generates a render.yaml file that specifies the build and startup commands required to run the application on the Render cloud platform.

The deployment module also ensures that previous project files are removed before copying new files. This prevents conflicts between different generated applications.

Once the deployment directory is prepared, the application can be deployed to the cloud hosting platform and accessed through a web browser.

3.2.7 Iterative Refinement

Iterative refinement is an important component of the Automated Application Builder system. The purpose of this stage is to improve the reliability and correctness of the generated application code by repeatedly validating and refining the output generated by the Large Language Model.

Large Language Models can generate high-quality code; however, the generated output may sometimes contain structural inconsistencies, missing elements, or formatting issues. To address these challenges, the system applies an iterative validation and refinement process. In this process, the generated code is analyzed using a set of predefined validation rules. If the generated output does not satisfy the validation requirements, corrective actions are applied to refine the code.

The validation module examines several aspects of the generated application, including the

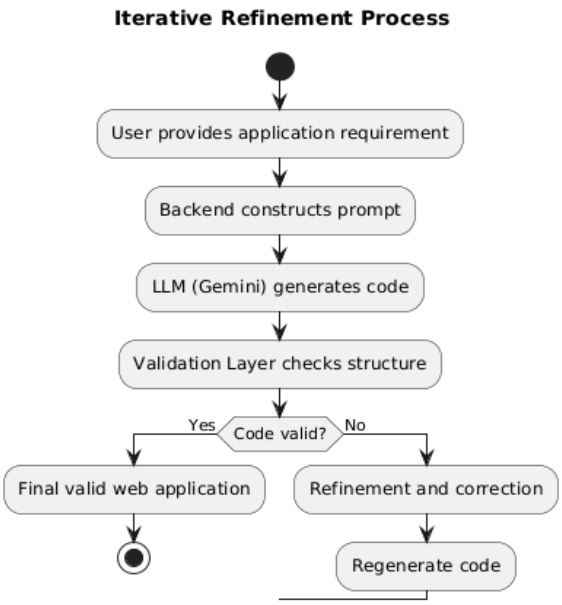


Fig. 3.1 Iterative Refinement Process

HTML structure, JavaScript logic, and code organization. If errors such as missing HTML tags, empty sections, or invalid navigation links are detected, the system applies modifications to correct these issues. For example, missing image references may be replaced with placeholder components, and unsafe navigation links may be removed to prevent unwanted page reloads.

Another important aspect of iterative refinement is ensuring that the generated application maintains visual consistency and functional completeness. The system ensures that each section of the webpage contains meaningful content and that the user interface remains responsive and interactive. If certain components are incomplete, the system replaces them with predefined

placeholder elements that maintain the visual layout of the page.

The iterative refinement process continues until the generated application satisfies all validation conditions. This process ensures that the generated application is structurally valid and suitable for execution or deployment.

The use of iterative refinement significantly improves the overall reliability of AI-generated applications. Instead of accepting the raw output generated by the AI model, the system performs additional verification and refinement steps that ensure the final output is functional and well-structured

3.3 Proposed Architecture

The proposed system architecture is designed to support automated generation of full-stack web applications using Large Language Models. The architecture follows a modular design that separates different responsibilities into independent components. This modular approach improves maintainability, scalability, and flexibility of the system.

The system architecture consists of several major modules including the frontend interface, backend processing module, AI automation engine, validation layer, and deployment module. Each module performs a specific function within the overall application generation pipeline.

The frontend interface acts as the user interaction layer where users submit their application requirements. The backend processing module receives these requirements and prepares structured prompts that guide the AI model during code generation. The AI automation engine generates application code based on the provided prompt, while the validation layer ensures that the generated code satisfies predefined structural rules.

Once the generated code passes validation, the backend organizes the code into a structured project directory containing the required frontend files, backend scripts, and configuration files. Finally, the deployment module prepares the generated application for hosting on cloud platforms.

The modular architecture ensures that each component can be updated or modified independently without affecting the entire system. This design also allows the system to integrate additional features such as advanced validation mechanisms or support for additional programming frameworks in the future.

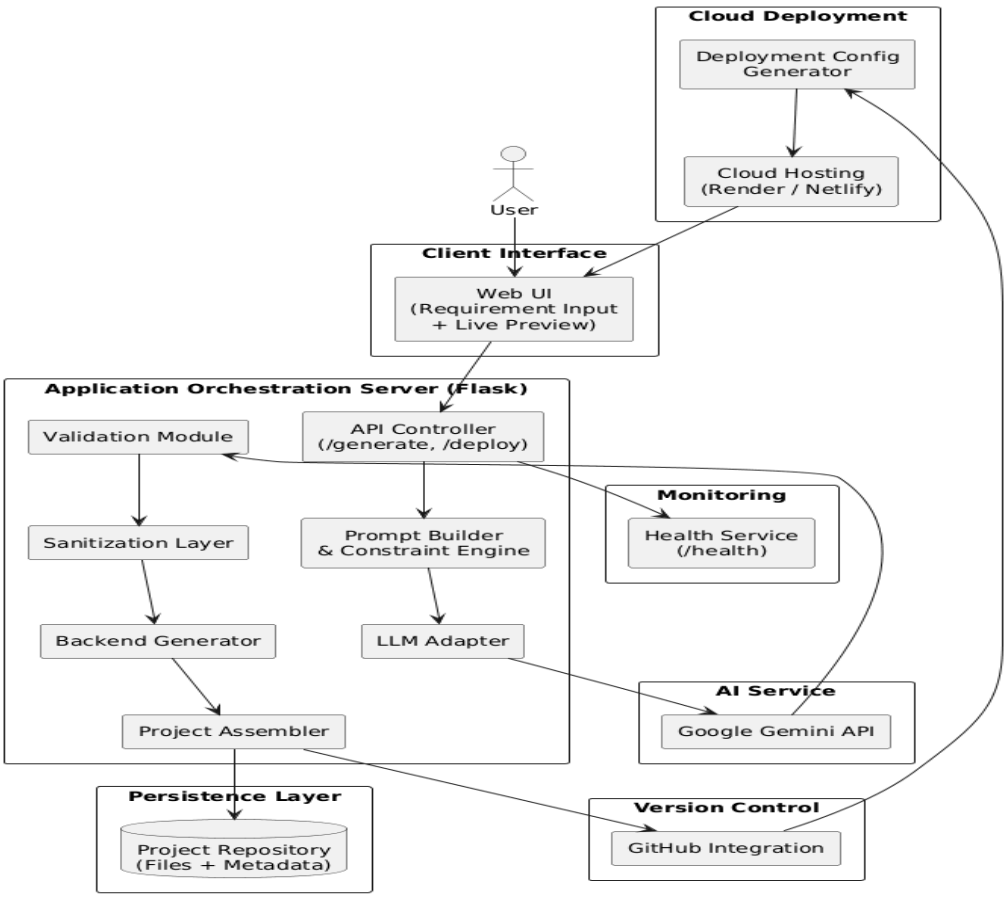


Fig 3.2 Architecture of Automated Application Builder using LLMs

3.3.1. Frontend Interface Module

The frontend interface module serves as the user interaction layer of the Automated Application Builder system. It provides a graphical interface through which users can describe the application they want to generate. The interface is designed to be simple and user-friendly so that users with minimal programming knowledge can interact with the system easily.

Through the frontend interface, users can specify various parameters related to the desired application. These parameters typically include the project name, application type, application description, and preferred technology stack. The interface collects this information and sends it to the backend server using HTTP requests.

The frontend also provides visualization features that allow users to view the generated application code and preview the generated web interface. This live preview capability enables users to verify whether the generated application meets their requirements before proceeding with deployment.

In addition to collecting user input, the frontend interface also handles communication with the backend server through RESTful API endpoints. These API calls allow the frontend to request

code generation, check system status, and retrieve generated application files.

The design of the frontend interface focuses on usability and responsiveness. The interface adapts to different screen sizes and devices, ensuring that users can interact with the system from desktop computers, tablets, or mobile devices.

3.3.2. Backend Processing Module

The backend processing module acts as the central control component of the system. It is implemented using the Flask web framework and is responsible for coordinating all major system operations including request processing, prompt construction, AI communication, validation, and project file generation.

When the frontend interface sends a request to generate an application, the backend receives the request through a REST API endpoint. The backend then extracts relevant information from the request payload and processes the data to prepare a structured prompt for the AI model.

The backend module also manages communication with the Google Gemini API. It sends the constructed prompt to the AI model and receives the generated code as a response. After receiving the generated output, the backend performs additional processing steps such as cleaning the response, extracting code segments, and validating the generated content.

Another important function of the backend module is managing project files and directories. The backend creates a project directory for each generated application and stores the generated HTML, CSS, JavaScript, and backend files in the appropriate locations.

The backend module also provides several API endpoints that support system functionality. These endpoints allow users to generate applications, deploy projects, test AI connectivity, and check system health status.

By acting as the central coordinator of the system, the backend processing module ensures smooth communication between different system components.

3.3.3. AI Automation Engine

The AI automation engine is the core component responsible for generating application code based on user requirements. In the proposed system, this component is implemented using the Google Gemini Large Language Model.

The AI automation engine receives structured prompts generated by the backend processing

module. These prompts contain detailed instructions that describe the desired application structure, design requirements, and functional behavior.

After receiving the prompt, the AI model analyzes the input and generates the corresponding application code. The generated output typically includes the complete HTML document along with embedded CSS and JavaScript code required to implement the application interface and functionality.

The AI model is capable of generating complex user interface components such as navigation bars, responsive layouts, interactive buttons, and content sections. The generated code follows modern web development practices and uses semantic HTML elements for better readability and maintainability.

One of the key advantages of using an AI automation engine is its ability to generate large amounts of code quickly and efficiently. Tasks that would normally require hours of manual coding can be completed within seconds using AI-generated code.

The AI automation engine therefore plays a crucial role in enabling automated full-stack web application generation.

3.3.4. Validation Layer

The validation layer is responsible for verifying the structural correctness and reliability of the generated application code. Since AI-generated code may occasionally contain errors or inconsistencies, the validation layer ensures that the generated output satisfies predefined development rules before the application is deployed.

The validation process includes multiple checks that analyze different aspects of the generated code. These checks verify the presence of required HTML elements, detect empty content sections, and ensure that the generated code follows proper formatting and structural conventions.

To evaluate the correctness of the generated code, the system computes a **validation accuracy metric**, which represents the proportion of successfully validated rules.

$$Validation\ Accuracy = \frac{R_{valid}}{R_{total}} \times 100$$

where:

- R_{valid} = number of validation rules satisfied
- R_{total} = total number of validation rules applied

This metric helps measure how well the generated code satisfies structural constraints.

The validation layer also applies security-related rules to prevent unsafe behaviors in the generated application. For example, navigation links that may cause unwanted page reloads are

removed, and image references are replaced with placeholder elements to avoid dependency on external resources.

Another important validation step involves verifying the presence of required style and script blocks. The completeness of required blocks can be expressed using the **component completeness ratio**:

$$Component\ Completeness = C\_present/C\_required\ \times 100$$

where:

- $C_{present}$ = number of required components present (HTML, CSS, JavaScript blocks)
- $C_{required}$ = total number of required components

The system also evaluates **section integrity**, ensuring that generated sections contain valid content rather than empty containers.

$$Section\ Integrity = S\_nonempty/S\_total\ \times 100$$

where:

- $S_{nonempty}$ = number of sections containing meaningful content
- S_{total} = total number of sections generated in the application

If the validation process detects errors or missing components, corrective actions are applied to refine the generated code. This ensures that the final application output is structurally valid and ready for execution.

By integrating automated validation metrics and rule-based correction mechanisms, the system improves the reliability of AI-generated web applications and reduces the likelihood of runtime errors during deployment.

$$Deployment\ Success\ Rate = A\_successful/A\_generated\ \times 100$$

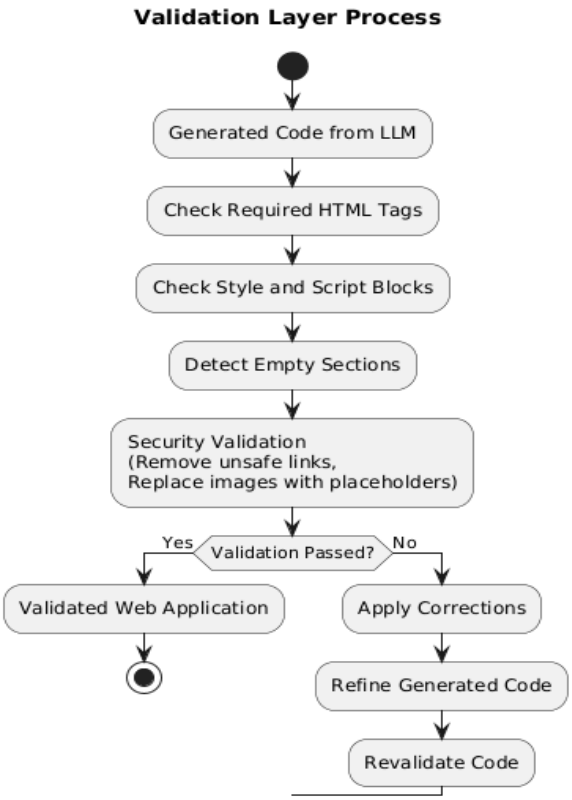


Fig 3.3 Validation process

3.4 System Requirements

Hardware Requirements:

Processor: Intel Core i5 or higher

RAM: Minimum 8 GB

Storage: Minimum 256 GB available disk space

Network: Stable internet connection for API communication

These hardware requirements provide sufficient computing resources to run the backend server, communicate with AI models, and manage generated project files.

Software Requirements:

- Python – 3.10.2
- Pandas – 2.2.1
- Matplotlib – 3.7.1
- Flask – 3.0.2
- Pytorch - 2.15.0
- Translate – 3.6.1

Operating System: Windows, Linux, or macOS

Programming Language: Python

Web Framework: Flask

AI Model: Google Gemini Large Language Model

Database (Optional): MongoDB Atlas

Deployment Platform: Render Cloud Hosting

Development Tools: Visual Studio Code or similar IDE

The software stack used in the system provides a lightweight and flexible environment for developing and running the Automated Application Builder. Python and Flask provide a simple yet powerful backend framework, while the Gemini AI model provides advanced code generation capabilities.

The integration of these technologies enables the system to automate the process of generating, validating, and deploying web applications.

Python:

Python is a high-level, interpreted programming language known for its readability, simplicity, and extensive ecosystem of libraries and frameworks. Developed in the late 1980s by Guido van Rossum, it has grown to become one of the most popular languages for software development, data analysis, and machine learning. Python's design philosophy emphasizes code clarity and conciseness, enabling developers to write efficient and maintainable code. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Its rich standard library and vibrant community foster rapid development and innovation across various domains. Python continues to drive breakthroughs in scientific computing and artificial intelligence.

Flask:

Flask is a lightweight and flexible web framework for building web applications and APIs in Python. It offers essential features for web development, including routing, request handling, templating, and session management, while remaining simple and easy to understand. Flask's minimalistic design and extensive ecosystem of extensions enable developers to create web applications quickly and efficiently, making it a popular choice for projects of all sizes.

Pytorch:

PyTorch is an open-source deep learning framework renowned for its dynamic computational graph, which provides exceptional flexibility and intuitive control for building, training, and deploying neural networks. Designed with a Pythonic interface, it integrates seamlessly with Python libraries, facilitating rapid prototyping and experimentation. Its automatic differentiation

feature simplifies the implementation of complex models by handling backpropagation efficiently. Moreover, PyTorch supports distributed training and GPU acceleration, enabling efficient scaling of models to handle large datasets. The framework's modular architecture, along with extensive libraries like TorchVision and TorchText, provides ready-to-use tools and pre-built models that accelerate development across various domains. Its active community and comprehensive documentation further empower researchers and developers to innovate and solve complex problems in artificial intelligence, making PyTorch a robust and versatile tool for both cutting-edge research and production-level applications.

CHAPTER 4

SYSTEM DESIGN

1.1 System Architecture

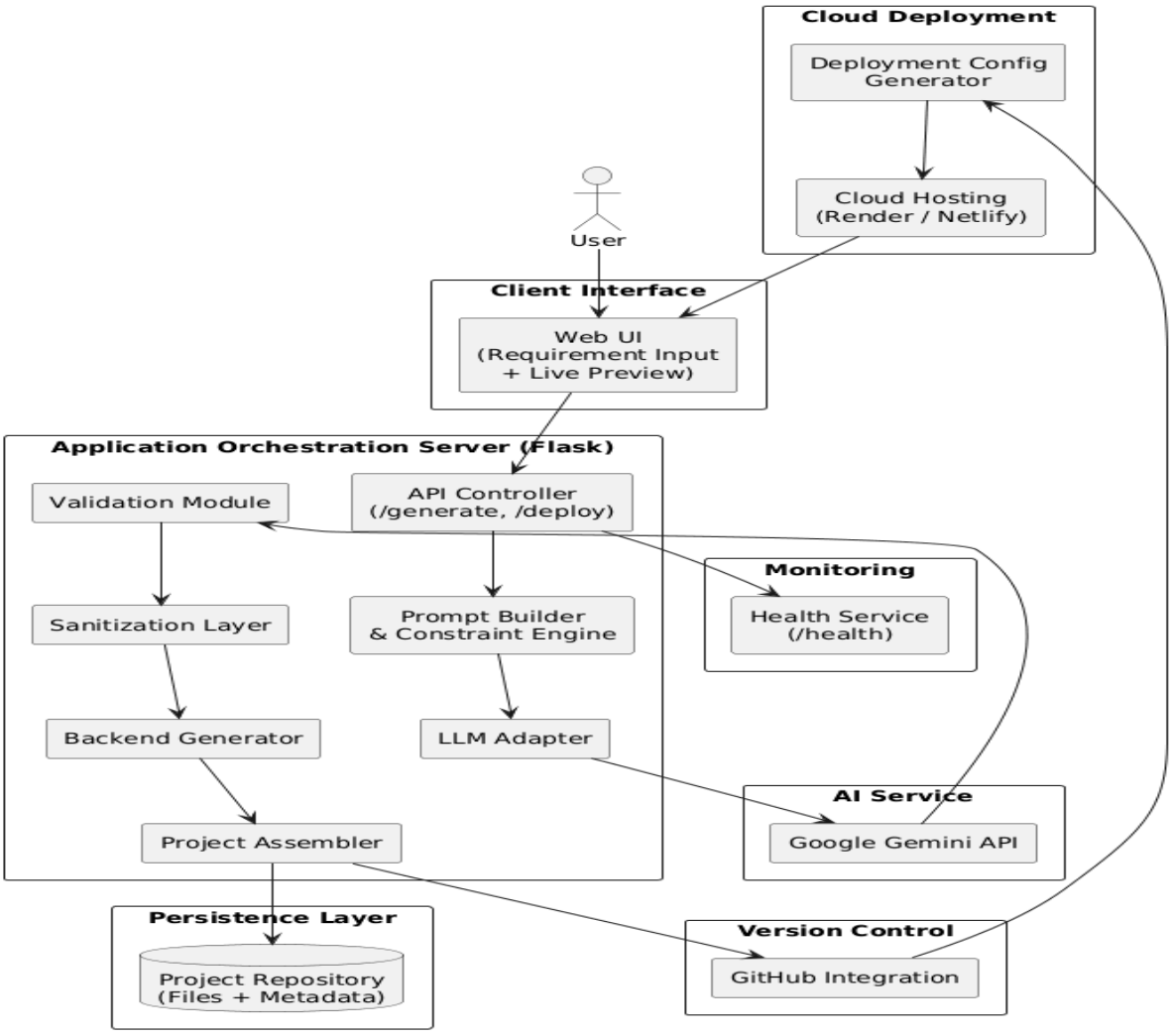


Fig 4.1 Architecture of the Automated Web Application Builder

The system architecture of the Automated Web Application Builder is designed to automate the process of generating full-stack web applications from natural language requirements. The architecture consists of several modules that work together to process user input, generate application code using AI, validate the generated output, and prepare the application for deployment.

The major components of the system architecture include:

• User Interface

• Backend Processing Server
• Prompt Processing Module
• AI Code Generation Engine
• Validation Layer
• Project Generation Module
• Deployment Module

The user interacts with the system through a web interface where they provide application requirements such as the project name, description, and technology stack. These inputs are transmitted to the backend server through REST API requests.

The backend server processes the user input and constructs a structured prompt that is sent to the Large Language Model. The AI model analyzes the prompt and generates the application code, which includes the HTML structure, CSS styling, and JavaScript functionality.

Once the code is generated, the backend server performs several validation checks to ensure that the generated application follows predefined structural and security rules. The validation module verifies that the generated code contains required HTML elements, script blocks, and style definitions.

After successful validation, the system organizes the generated files into a structured project directory containing the frontend files, backend server code, and deployment configuration files.

Finally, the deployment module prepares the generated project for hosting on a cloud platform. The deployment process ensures that the generated application can be executed in a production environment.

This architecture ensures that the entire process of application development—from requirement input to deployment—is automated and efficient.

1.1 Flow Chart Diagram

The flowchart diagram illustrates the sequence of steps involved in generating a web application using the Automated Application Builder. The flowchart provides a visual representation of the system workflow and shows how the different modules interact with each other.
The process begins when the user enters application requirements through the frontend interface. These requirements include details about the application type, description, and technology stack.
After receiving the user input, the backend server processes the request and constructs a

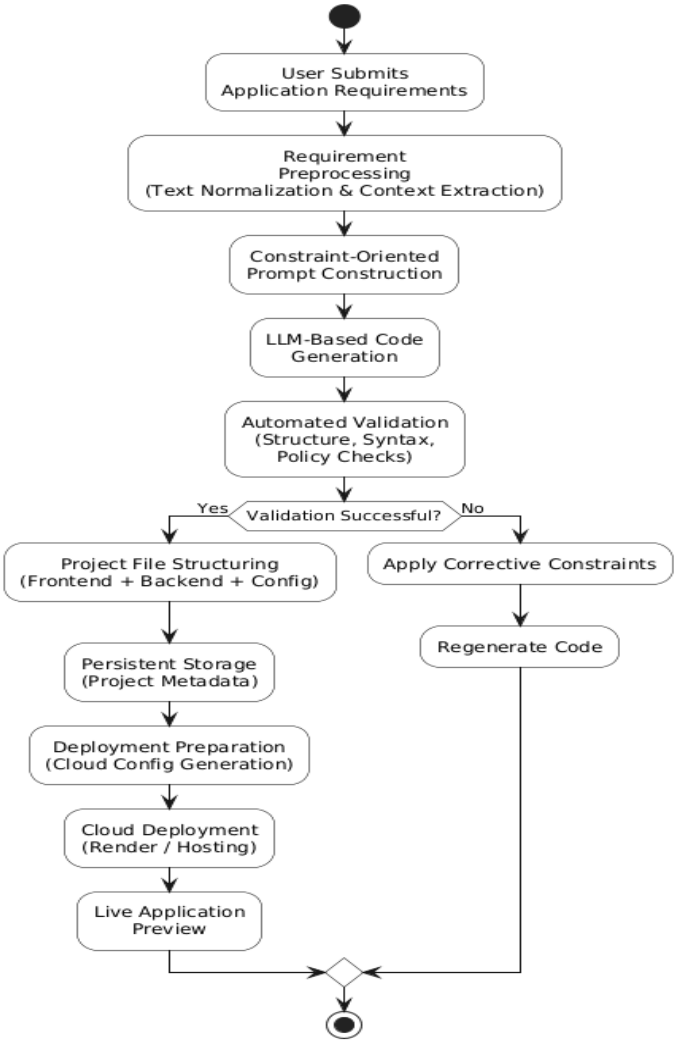
structured prompt that is sent to the Large Language Model. The AI model analyzes the prompt and generates the corresponding application code.

The generated code is returned to the backend server, where it undergoes validation to ensure that the generated output is structurally correct. The validation process checks for required HTML tags, script blocks, and section completeness.

If validation fails, the system applies corrective modification an iterative refinement process. If the code passes validation, the system organizes the generated files into a structured project directory.

Finally, the project is prepared for deployment, allowing the generated application to be hosted on a cloud platform.

The flowchart therefore represents the complete pipeline of the automated web application



generation process.

Fig 4.2 Flow Chart Diagram

1.2 UML Diagrams

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components- a Metamodel and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

Goals:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns, and components.
7. Integrate best practices.

1.2.1 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show

what system functions are performed for which actor. Roles of the actors in the system can be depicted.

An effective use case diagram can help your team discuss and represent:

- Scenarios in which your system or application interacts with people, organizations, or external system
- Goals that your system or application helps those entities (known as actors(achieve
- The scope of your system

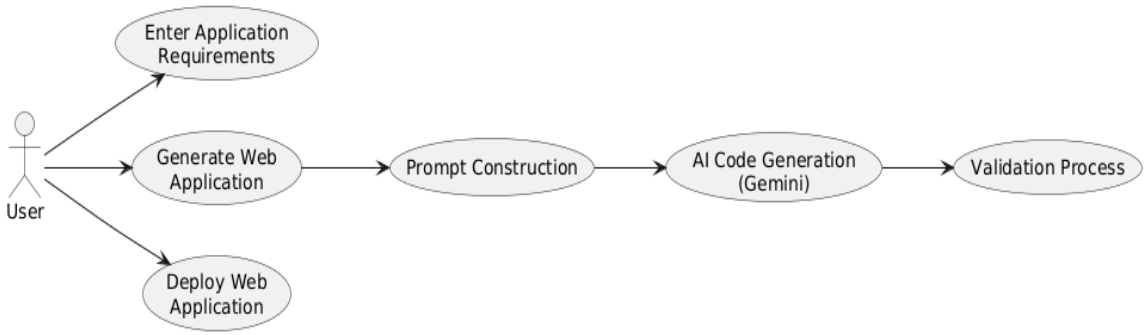


Fig 4.3 Use Case Diagram

1.2.2 Class Diagram:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

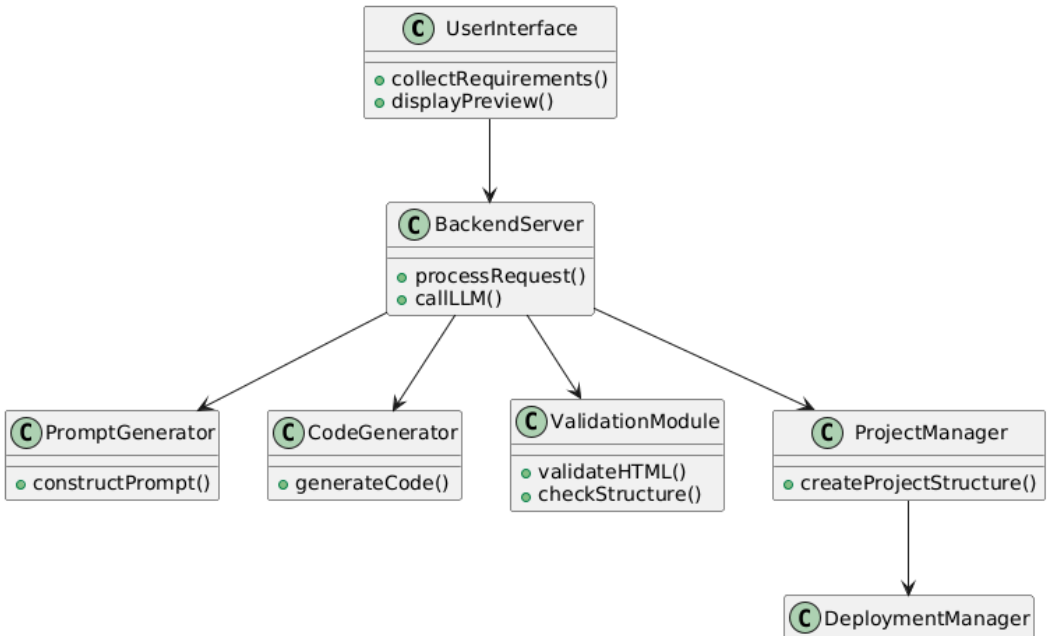


Fig 4.4 Class Diagram

1.2.3 Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

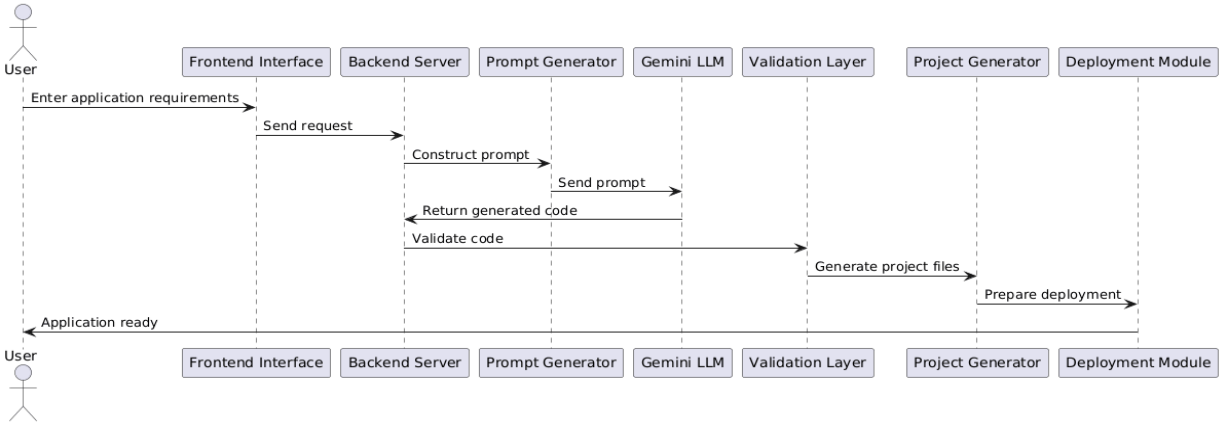


Fig 4.5 Sequence Diagram

1.2.4 Activity Diagram:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration, and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

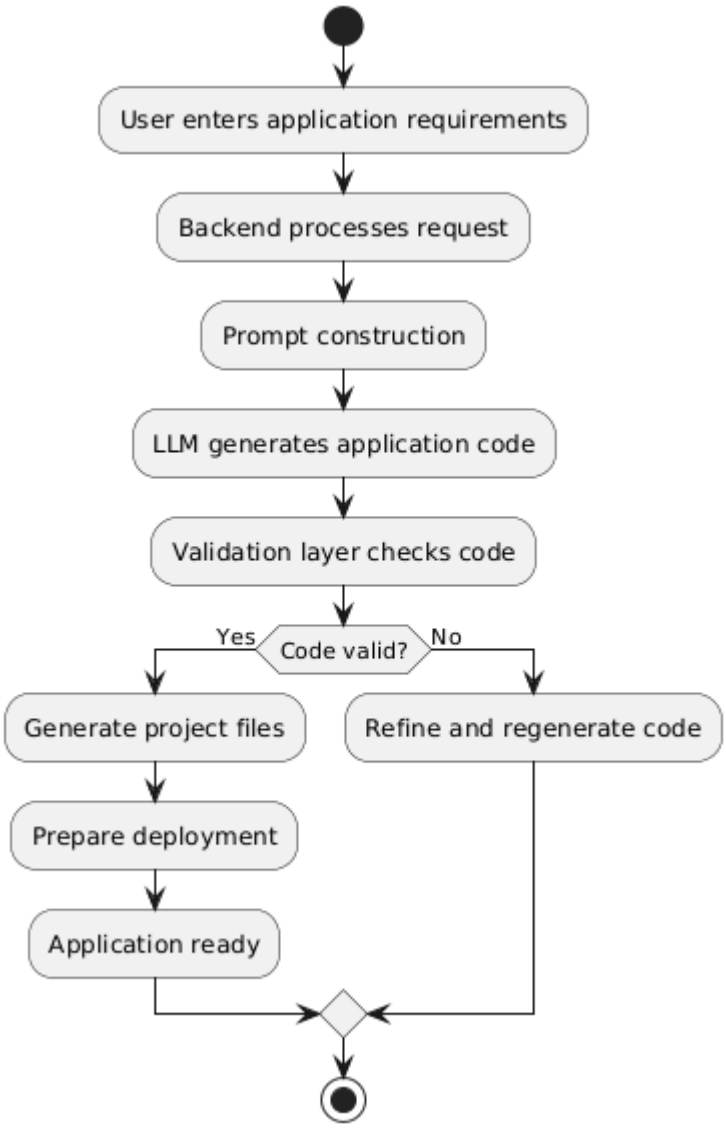


Fig 4.6 Activity diagram

CHAPTER 5
DEPLOYMENT

Deployment is an essential phase in the development lifecycle of any software system. It involves preparing the application for execution in a real-world environment and ensuring that the system operates reliably when accessed by users. In the context of the Automated Web Application Builder, deployment refers to the process of hosting the backend server, integrating the AI code generation engine, managing data storage, and enabling the generated web applications to run on a cloud platform.

The deployment architecture of the proposed system is designed to ensure scalability, reliability, and accessibility. The system uses Python as the core programming language and the Flask web framework to implement the backend server. The backend server communicates with the Google Gemini Large Language Model API to generate application code dynamically based on user requirements.

To support persistent data storage and project management, the system can integrate with MongoDB Atlas, a cloud-based NoSQL database platform. MongoDB Atlas enables secure and scalable data storage that can be accessed by the backend server during runtime.

The final stage of deployment involves hosting the application on a cloud platform. In this project, the generated applications are deployed using the Render cloud hosting platform. Render provides an environment for running web services and deploying backend applications with minimal configuration.

The deployment architecture therefore combines multiple technologies to create a robust and automated environment for generating and executing web applications.

Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** − You can actually sit at a Python prompt and interact with the

interpreter directly to write your programs.

- **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

- **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

- Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Smalltalk, and Unix shell and other scripting languages.
- Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).
- Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

Python's features include −

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** − Python provides interfaces to all major commercial databases.

- **GUI Programming** − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below −

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

Python is available on a wide variety of platforms including Linux and Mac OS X. Let's understand how to set up our Python environment.

Getting Python

The most up-to-date and current source code, binaries, documentation, news, etc., is available on the official website of Python https://www.python.org.

Windows Installation

Here are the steps to install Python on Windows machine.

- Open a Web browser and go to https://www.python.org/downloads/.
- Follow the link for the Windows installer python-XYZ.msifile where XYZ is the version you need to install.
- To use this installer python-XYZ.msi, the Windows system must support Microsoft Installer 2.0. Save the installer file to your local machine and then run it to find out if your machine supports MSI.
- Run the downloaded file. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you are done.

The Python language has many similarities to Perl, C, and Java. However, there are some definite differences between the languages.

5.2 Flask Web Framework & Google Gemini LLM

The backend of the Automated Web Application Builder is implemented using the Flask web framework. Flask is a lightweight and flexible web framework for Python that is widely used for building web applications and RESTful APIs.

Flask follows a micro-framework architecture, which means it provides the essential components required to build web applications without imposing strict project structures. This flexibility allows developers to design custom application architectures based on project requirements.

In the proposed system, Flask acts as the backend server that handles communication between the frontend interface and the AI code generation engine. The Flask application exposes several API endpoints that allow the frontend to interact with the backend system.

For example, the backend provides endpoints that allow users to submit application requirements, generate application code, test AI connectivity, and deploy generated applications. These API endpoints enable smooth communication between different system components.

Flask also supports middleware components such as Flask-CORS, which enables cross-origin resource sharing between the frontend interface and the backend server. This feature ensures that the frontend application can securely communicate with the backend server.

Another major component integrated into the backend system is the Google Gemini Large Language Model. Gemini is an advanced AI model capable of understanding natural language input and generating structured programming code.

In this project, the Gemini model is used as the AI code generation engine. The backend constructs detailed prompts based on user requirements and sends them to the Gemini API. The AI model analyzes the prompt and generates the corresponding application code, including HTML structure, CSS styling, and JavaScript functionality.

The integration of Flask with the Gemini API allows the system to automate the process of generating web applications. The backend server manages prompt construction, API communication, and response processing, ensuring that the generated code follows predefined structural guidelines.

By combining the lightweight architecture of Flask with the powerful capabilities of the Gemini Large Language Model, the system can efficiently generate functional web applications from natural language descriptions.

.

5.3 MongoDB Atlas & Cloud Deployment on Render

Render is a modern cloud platform designed to simplify the deployment and hosting of web applications, APIs, and backend services. It provides a fully managed infrastructure that allows developers to deploy applications without manually configuring servers or managing complex deployment pipelines.

One of the major advantages of Render is its ability to support multiple types of services including web services, static websites, background workers, and databases. This flexibility allows developers to deploy both frontend and backend components of an application using the same platform.

In the Automated Web Application Builder project, Render is used to host the backend Flask application and execute the generated web applications. The backend server processes user requests, communicates with the Gemini Large Language Model API, validates generated code, and prepares the application for deployment. By hosting the backend server on Render, the system becomes accessible to users through the internet.

Render provides a simple deployment workflow that integrates directly with Git repositories such as GitHub. When code is pushed to the repository, Render automatically builds and deploys the application using predefined configuration settings. This process is known as continuous deployment, and it allows developers to update their applications quickly without manual intervention.

Another important feature of Render is its support for environment variables. Environment variables allow developers to securely store sensitive information such as API keys, database connection strings, and configuration parameters. In this project, environment variables are used to store the Google Gemini API key and other configuration settings required by the backend server.

Render also provides automatic scaling and monitoring capabilities. The platform monitors application performance and ensures that services remain available even when traffic increases. This makes Render suitable for hosting scalable web applications.

For Python-based applications such as Flask servers, Render automatically installs the required dependencies using a requirements file. The platform then runs the application using a specified startup command, such as launching the Flask server using a WSGI server like Gunicorn.

Another important advantage of Render is its simplified configuration through deployment configuration files such as render.yaml. These configuration files define the build commands, start commands, environment settings, and service configurations required to run the application.

In the Automated Web Application Builder system, the deployment process typically follows these steps:

- The backend server generates a project structure containing frontend files, backend scripts, and configuration files.
- The deployment module prepares the project for cloud hosting by creating the necessary configuration files.
- The project is uploaded to the Render deployment environment.
- Render installs dependencies and builds the application automatically.
- The application server starts and becomes accessible through a public URL.

Once deployed, users can access the generated web application through the cloud-hosted server. This enables real-time testing and demonstration of the generated application.

Render also supports HTTPS security by automatically providing SSL certificates for deployed applications. This ensures secure communication between users and the deployed application.

Because of its simplicity, scalability, and built-in deployment automation, Render serves as an effective platform for hosting the Automated Web Application Builder and executing the generated web applications in a production environment.

The Automated Web Application Builder requires a reliable data storage system and a cloud hosting platform to ensure that generated applications can be stored and executed effectively. To achieve this, the system integrates MongoDB Atlas for data storage and Render for cloud deployment.

MongoDB Atlas is a cloud-based NoSQL database service that provides scalable and secure data storage. Unlike traditional relational databases, MongoDB stores data in flexible JSON-like documents, which makes it suitable for modern web applications.

In the proposed system, MongoDB Atlas can be used to store project metadata, user requests, generated code snippets, and deployment information. This allows the system to maintain records of generated applications and manage project data efficiently.

MongoDB Atlas provides several advantages including automatic scaling, built-in security features, and global cloud availability. The database can be easily integrated with Python applications using MongoDB drivers, enabling seamless communication between the backend server and the database.

For hosting the backend server and generated applications, the system uses the Render cloud hosting platform. Render is a modern cloud platform that allows developers to deploy web services, static sites, and backend applications with minimal configuration.

Render provides several features that simplify application deployment, including automatic builds, continuous deployment, and secure environment variable management. Developers can deploy applications directly from their project repositories, and the platform automatically builds and runs the application.

In the proposed system, Render is used to host the backend Flask application and execute generated web applications. Deployment configuration files such as `render.yaml` specify the commands required to install dependencies and start the backend server.

By combining MongoDB Atlas for data storage and Render for application hosting, the system ensures that generated web applications can be stored, managed, and deployed in a scalable cloud environment.

This deployment architecture allows the Automated Web Application Builder to operate as a complete end-to-end system capable of generating, validating, and hosting web applications automatically.

CHAPTER 6
SYSTEM TESTING

6.1 Purpose of Testing

The purpose of testing is to verify that the Automated Web Application Builder performs all required functions accurately and efficiently. Testing ensures that each component of the system works as expected and that the entire system behaves correctly when integrated together.

One of the main objectives of testing is to detect errors that may occur during code generation, validation, or deployment. Because the system relies on AI-generated code, there is a possibility that the generated output may contain structural inconsistencies or missing components. Testing helps identify such issues and ensures that appropriate validation mechanisms are applied.

Another important purpose of testing is to evaluate system performance and reliability. The system must be capable of handling user requests, generating application code, validating outputs, and preparing projects for deployment without delays or failures.

Testing also helps ensure that the system provides a smooth user experience. The frontend interface must correctly capture user inputs, display generated code, and present application previews without errors.

Additionally, testing verifies that the backend server correctly communicates with the Gemini AI model and processes the generated responses effectively.

By performing comprehensive testing, developers can ensure that the Automated Web Application Builder operates reliably and produces functional web applications.

6.2 Types of Testing

6.2.1 Unit Testing

Unit testing focuses on verifying the functionality of individual components or modules of the system. Each module is tested independently to ensure that it performs the expected operations correctly.

In the Automated Web Application Builder, unit testing is applied to individual functions within the backend server. These functions include prompt construction, API communication with the Gemini model, validation functions, and project file generation.

For example, the prompt generation function is tested to ensure that it correctly constructs structured prompts based on user input. Similarly, validation functions are tested to verify that they correctly detect missing HTML tags, empty sections, or invalid navigation links.

Unit testing ensures that each module performs correctly before it is integrated with other system components. This reduces the likelihood of errors during later stages of system development.

6.2.2 Integration Testing

Integration testing focuses on verifying the interactions between different modules of the system. While unit testing ensures that individual components work correctly, integration testing ensures that these components communicate properly when combined together.

In this project, integration testing is used to verify communication between the frontend interface, backend server, and AI code generation engine.

For example, when a user submits application requirements through the frontend interface, the system sends the request to the backend server through an API endpoint. The backend server processes the request, constructs a prompt, and sends the prompt to the Gemini AI model. The generated code is then returned to the backend server for validation.

Integration testing verifies that this entire workflow operates correctly without communication failures or data inconsistencies.

6.2.3 Functional Testing

Functional testing evaluates whether the system performs its intended functions according to the specified requirements. This type of testing focuses on verifying system behavior from the user's perspective.

In the Automated Web Application Builder, functional testing ensures that the system can successfully generate web applications based on user input. The system is tested using different types of application descriptions to verify that the AI model generates appropriate code.

Functional testing also verifies that generated applications contain the required components such as navigation menus, content sections, CSS styling, and JavaScript functionality.

Additionally, functional testing ensures that the generated applications operate correctly when executed in a web browser.

6.2.4 System Testing

System testing evaluates the complete system as a whole. It verifies that all integrated components operate together correctly and that the system meets overall performance and functionality requirements.

In the proposed system, system testing involves evaluating the entire application generation pipeline, starting from user input and ending with deployment of the generated web application.

System testing verifies that the frontend interface correctly captures user requirements, the backend server processes requests properly, the AI model generates valid application code, and the validation layer ensures structural correctness.

The testing process also ensures that generated project files are correctly organized and that the application can be deployed successfully to the cloud environment.

6.2.5 White Box Testing:

White box testing focuses on examining the internal logic and structure of the program. In this testing method, developers analyze the internal code of the system to ensure that the program logic functions correctly.

White box testing is applied to the backend server functions implemented using Python and Flask. Developers analyze the control flow of functions such as code validation, project file generation, and API request handling.

This type of testing ensures that the internal logic of the program follows correct execution paths and that all conditional statements and loops operate as expected.

White box testing also helps identify logical errors, inefficient code structures, and potential security vulnerabilities within the system.

6.2.6 Black Box Testing:

Black box testing evaluates the system based on its external behavior without examining the internal program structure. In this testing approach, the tester interacts with the system as a user and verifies whether the system produces the expected outputs for given inputs.

In the Automated Web Application Builder, black box testing is performed by providing different application descriptions through the frontend interface and observing the generated output.

The tester verifies whether the system successfully generates valid application code and whether the generated applications function correctly when executed.

Black box testing helps ensure that the system behaves correctly from the user's perspective and that the application builder provides accurate and reliable results.

CHAPTER 7

RESULTS & DISCUSSIONS

7.1 Accuracy and Loss

The performance of the Automated Web Application Builder is evaluated using **functional accuracy**, which measures the percentage of generated applications that execute successfully with integrated frontend components, backend services, and deployment configurations.

A generated application is considered successful when the following conditions are satisfied:

- The frontend interface renders correctly in the browser
- Backend API services operate properly
- The generated project structure is valid
- The application can be deployed successfully to the cloud platform

The functional accuracy is calculated using the following formula:

$$Accuracy = \frac{N_s}{N_t} \times 100$$

Where:

- N_s= Number of successfully executed applications
- N_t= Total number of generated applications

Experimental results indicate that the proposed system achieves an **overall functional accuracy of approximately 86%**. This demonstrates that the integration of **structured prompt engineering, validation mechanisms, and automated deployment support** significantly improves the reliability of AI-generated web applications.

Batch08IEEE_Conference_FBH

The high accuracy is mainly due to the system's ability to maintain **architectural consistency across frontend, backend, and database components** while performing validation-driven refinement before deployment.

Approach	Frontend	Backend	Deployment	Overall Accuracy
Image-based Generation	70	Not Applicable	Not Applicable	62
Sketch-based Generation	75	Low	Not Applicable	68
Template Low-Code Tools	80	Partial	Limited	74
Generic LLM Models	88	82	Partial	83
Proposed LLM System	**90**	**87**	**Yes**	**86**

Fig. 7.1 Accuracy value Compare to Existing Methodology

7.2 OUTPUT OF PROPOSED SYSTEM

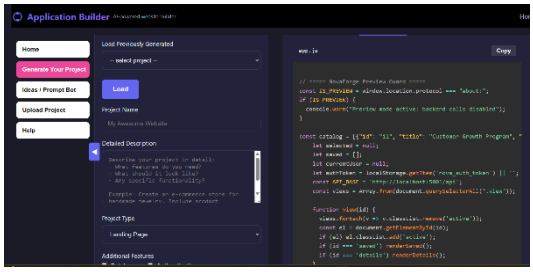
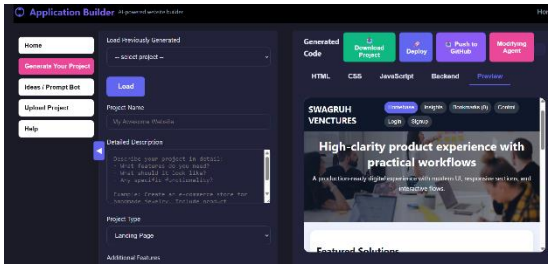


Fig.7.2,7.3 Code Generation & Preview

Fig 7.4,7.5 Result Generated



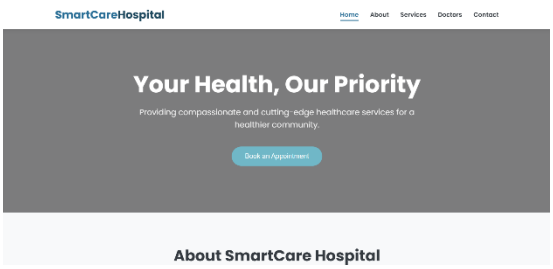
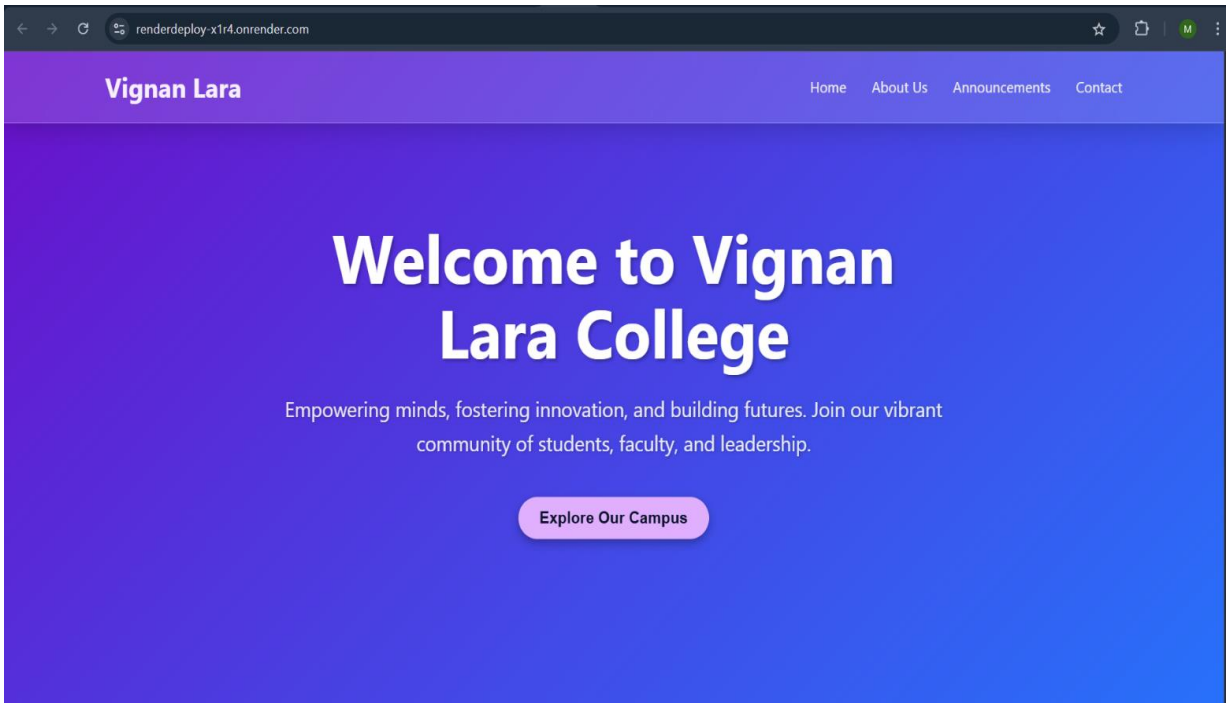


Fig. 7.6 Render Deployed Project



CHAPTER 8
CONCLUSION AND FUTURE WORK

This work presented an AI-based automated web application builder designed to simplify and accelerate full-stack web development through natural language interaction. The proposed system effectively transforms user requirements into complete and deployable web applications by integrating AI-driven code generation, backend coordination, validation mechanisms, and cloud deployment within a unified framework. Experimental results demonstrate that the system can generate functional applications with high practical accuracy while providing real-time previews and production-like cloud deployment for effective evaluation. By reducing manual coding effort and technical complexity, the proposed approach is well suited for academic projects, rapid prototyping, and early-stage application development. Overall, this study demonstrates that Large Language Models, when combined with structured workflows and deployment-aware system design, can serve as practical and reliable tools for end-to-end web application automation.

While the proposed system successfully automates key aspects of full-stack web application development, there is scope for further improvement. Future efforts can focus on enhancing backend flexibility, strengthening security measures, and expanding compatibility with additional frameworks and deployment platforms. Incorporating feedback-based refinement mechanisms may also improve consistency and adaptability over time.

A. Current Limitations

At present, the system relies on an externally hosted pre trained Large Language Model, which limits fine-grained control over the generation process. As a result, small variations in structure or interface layout may appear even for similar inputs. The platform is primarily designed for individual use and does not yet provide built-in support for collaborative development. Moreover, advanced automated testing, security analysis, and performance optimization features remain limited.

B. Future Enhancements

Future improvements may include integrating automated testing and security evaluation tools to identify logical issues and potential vulnerabilities before deployment. Adaptive prompt refinement based on prior outputs and user feedback can help improve reliability and reduce unnecessary regeneration cycles. Enabling collaborative workflows through role-based access and version tracking would further enhance usability. Additionally,

combining generative models with rule-based constraints or domain-specific guidelines could offer better control over output quality while preserving flexibility. Such advancements would move the system closer to a scalable and practically deployable solution for AI-assisted web application development

CHAPTER 9

REFERENCES

1. S. Ratani, D. Phatta, D. Phalke, N. Varun, and A. Aher, “Web Sculptor: Generative AI Based Comprehensive Web Development Framework,” in Proc. IEEE Int. Conf. on Computing, Power, and Communication Technologies (IC2PCT), 2024.
2. V. H. Muthazhagu and S. B, “Exploring the Role of AI in Web Design and Development: A Voyage through Automated Code Generation,” in Proc. IEEE Int. Conf. on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE), 2024.
3. T. Kaluarachchi and M. Wickramasinghe, “A Systematic Literature Review on Automatic Website Generation,” Journal of Computer Lan guages, vol. 75, 2023.
4. T. Beltramelli, “pix2code: Generating Code from a Graphical User Interface Screenshot,” arXiv preprint arXiv:1705.07962, 2017.
5. S. Sonje, H. Dave, J. Pardeshi, and S. Chaudhari, “draw2code: AI-based Auto Web Page Generation from Hand-drawn Page Mock-up,” in Proc. IEEE Int. Conf. for Convergence in Technology (I2CT), 2022.
6. B. B. Adefris, A. B. Habtie, and Y. G. Taye, “Automatic Code Gener ation from Low-Fidelity Graphical User Interface Sketches Using Deep Learning,” in Proc. IEEE Int. Conf. on Information and Communication Technology for Development for Africa (ICT4DA), 2022.
7. C. Nikam, R. Keshervani, S. Shah, and J. Aghav, “Code Generation from Diagrams Using Neural Networks,” in Proc. Int. Conf. on Computational Intelligence, Springer, 2022.
8. J. Pacheco, S. Garbatov, and M. Goulao, “Improving Collaboration Efficiency Between UX/UI Designers and Developers in a Low-Code Platform,” in Proc. ACM/IEEE Int. Conf. on Model Driven Engineering Languages and Systems Companion (MODELS-C), 2021.
9. V. Bilgram and F. Laarmann, “Accelerating Innovation with Generative AI: AI-Augmented Digital Prototyping,” IEEE Engineering Management Review, 2023.
10. C. M. Novac et al., “Comparative Study of Applications Built Using Vue.js and React.js Frameworks,” in Proc. IEEE Int. Conf. on Engineer ing of Modern Electric Systems (EMES), 2021.
11. D. Pawade et al., “Automatic HTML Code Generation from Graphical User Interface Image,” in Proc. IEEE Int. Conf. on Recent Trends in Electronics, Information and Communication Technology (RTEICT), 2018.

12. Y. Xu et al., “Image2Emmet: Automatic Code Generation from Web User Interface Images,” Journal of Software: Evolution and Process, vol. 33, no. 8, 2021.
13. V. Jain et al., “Sketch2Code: Transformation of Sketches to UI in Real Time Using Deep Neural Network,” in Proc. IEEE, 2019.
14. T. A. Nguyen and C. Csallner, “Reverse Engineering Mobile Application User Interfaces with REMAUI,” in Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), 2015.
15. S. Natarajan and C. Csallner, “P2A: Converting Pixels to Animated Mobile Application User Interfaces,” in Proc. ACM Int. Conf. on Mobile Software Engineering and Systems (MOBILESoft), 2018.
16. G. Paolone et al., “Automatic Code Generation of MVC Web Applica tions,” Computers, vol. 9, no. 3, 2020.
17. J. Cheng, J. Li, and J. Chen, “Recognition Method of Interface Elements Based on Machine Learning,” in Proc. IEEE Int. Conf. on Intelligent Computing and Applications (ICAA), 2021.
18. K. Moran et al., “Machine Learning-Based Prototyping of Graphical User Interfaces for Mobile Apps,” IEEE Transactions on Software Engineering, 2018

APPENDIX

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import google.generativeai as genai
from dotenv import load_dotenv
import os
import pathlib
import shutil
import re

# ------------------------------
# Load Environment Configuration
# ------------------------------
load_dotenv()

app = Flask(__name__)
CORS(app)

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

# Configure Gemini Model
genai.configure(api_key=GEMINI_API_KEY)
model = genai.GenerativeModel("models/gemini-2.5-flash")

# Project directories
BASE_DIR = pathlib.Path(__file__).resolve().parent
GENERATED_ROOT = BASE_DIR / "generated_projects"
DEPLOY_TARGET = BASE_DIR / "deploy_target"

GENERATED_ROOT.mkdir(exist_ok=True)
DEPLOY_TARGET.mkdir(exist_ok=True)


# ------------------------------
# API: Generate Web Application
# ------------------------------
@app.route("/api/generate", methods=["POST"])
def generate_code():
    try:
        data = request.json
```

```
project_name = data.get("projectName", "Website")
project_type = data.get("projectType", "landing")
description = data.get("description", "")
tech_stack = data.get("techStack", ["HTML", "CSS", "JavaScript"])

# Prompt for LLM
prompt = f"""
Generate a production-ready single page web application.

Project Name: {project_name}
Type: {project_type}
Description: {description}

Requirements:
- Use semantic HTML structure
- Include responsive layout
- Add navigation sections (Home, About, Services, Contact)
- Use embedded CSS and JavaScript
"""

print(f"Generating project: {project_name}")

response = model.generate_content(prompt)
generated_code = response.text.strip()

# Extract CSS and JS blocks
css_code = extract_between_tags(generated_code, "<style>", "</style>")
js_code = extract_between_tags(generated_code, "<script>", "</script>")

# Apply validation rules
generated_code = enforce_no_base_tag(generated_code)
generated_code = enforce_no_images(generated_code)

# Save project files
safe_name = "".join(
    c if c.isalnum() or c in "-_" else "-"
    for c in project_name.strip()
)

project_dir = GENERATED_ROOT / safe_name

save_project_to_disk(
    project_dir,
    generated_code,
```

```
            css_code,
            js_code
        )

        return jsonify({
            "success": True,
            "projectName": project_name,
            "html": generated_code,
            "css": css_code,
            "js": js_code
        })

    except Exception as e:
        return jsonify({
            "success": False,
            "error": str(e)
        })


# ------------------------------
# Validation Functions
# ------------------------------
def enforce_no_images(html):
    """Replace <img> tags with placeholder"""
    placeholder = "<div class='image-placeholder'>ADD IMAGE</div>"
    return re.sub(r"<img[^>]*>", placeholder, html)


def enforce_no_base_tag(html):
    """Remove <base> tag"""
    return re.sub(r"<base[^>]*>", "", html)


# ------------------------------
# Extract CSS / JS blocks
# ------------------------------
def extract_between_tags(html, start_tag, end_tag):
    try:
        start = html.find(start_tag)
        end = html.find(end_tag)

        if start != -1 and end != -1:
            return html[start + len(start_tag):end].strip()
```

```
    except:
        pass

    return ""


# ------------------------------
# Save Generated Project
# ------------------------------
def save_project_to_disk(project_dir, html, css, js):

    project_dir.mkdir(parents=True, exist_ok=True)

    (project_dir / "index.html").write_text(html)
    (project_dir / "style.css").write_text(css)
    (project_dir / "app.js").write_text(js)


# ------------------------------
# API: Deploy Project
# ------------------------------
@app.route("/api/deploy", methods=["POST"])
def deploy_project():

    try:
        data = request.json
        project_name = data.get("projectName")

        project_dir = GENERATED_ROOT / project_name

        if not project_dir.exists():
            return jsonify({"success": False, "error": "Project not found"})

        # Clear old deployment
        for item in DEPLOY_TARGET.iterdir():
            if item.is_dir():
                shutil.rmtree(item)
            else:
                item.unlink()

        # Copy new project
        for item in project_dir.iterdir():
            dest = DEPLOY_TARGET / item.name
```

```
            if item.is_dir():
                shutil.copytree(item, dest)
            else:
                shutil.copy2(item, dest)

        return jsonify({
            "success": True,
            "message": "Project deployed"
        })

    except Exception as e:
        return jsonify({"success": False, "error": str(e)})


# ---------------------------------
# API: Health Check
# ---------------------------------
@app.route("/api/health", methods=["GET"])
def health_check():

    return jsonify({
        "status": "running",
        "service": "AI Web Application Builder"
    })


# ---------------------------------
# Run Server
# ---------------------------------
if __name__ == "__main__":

    print("AI Web Application Builder Backend Started")
    print("Server running on http://localhost:5000")

    app.run(debug=True, port=5000)
```